

EXP.NO: 8	Bankers Deadlock Avoidance Algorithm
DATE: 13/03/2026	

AIM:

To implement Banker's Deadlock Avoidance Algorithm in Linux.

ALGORITHM:

1. Read number of processes and resources.
2. Input Allocation, Maximum and Available matrices.
3. Calculate Need matrix using:

$$\text{Need} = \text{Max} - \text{Allocation}$$

4. Check whether resources can be allocated safely.
5. If all processes execute safely, print Safe Sequence.
6. Otherwise, declare system is in unsafe state.

CODE:

```
#include<stdio.h>
int main()
{
    int n = 5, m = 3, i, j, k;
    int alloc[5][3] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
    int max[5][3] = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
    int avail[3] = {3,3,2};
    int finish[5] = {0}, safe[5], need[5][3];
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];
    int count = 0;
    while(count < n)
    {
        for(i=0;i<n;i++)
        {
            if(finish[i] == 0)
            {
                int flag = 1;
                for(j=0;j<m;j++)
                {
```

```

        if(need[i][j] > avail[j])
        {
            flag = 0;
            break;
        }
    }

    if(flag)
    {
        safe[count++] = i;

        for(k=0;k<m;k++)
            avail[k] += alloc[i][k];

        finish[i] = 1;
    }
}

}

}

printf("Safe Sequence is:\n");
for(i=0;i<n;i++)
    printf("P%d ", safe[i]);
return 0;
}

```

OUTPUT:

```

[hp@localhost ~]$ gcc bankers_algorithm.c -o bankers
[hp@localhost ~]$ ./bankers
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

----- SYSTEM STATE -----
Allocation Matrix:      Max Matrix:      Need Matrix:      Available:
0 1 0                  7 5 3          7 4 3          3 3 2
2 0 0                  3 2 2          1 2 2
3 0 2                  9 0 2          6 0 0
2 1 1                  2 2 2          0 1 1
0 0 2                  4 3 3          4 3 1

System is in SAFE state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
[hp@localhost ~]$ ./bankers
Enter number of processes: 5
Enter number of resource types: 3
...
Enter Available Resources:
0 0 0
System is in UNSAFE state. No safe sequence possible.
[hp@localhost ~]$ █

```

RESULT:

Thus, Banker's Deadlock Avoidance Algorithm was implemented and executed successfully in Linux.